Redes Neuronales – E1 parte 1

Requerimientos para realizar el ejercicio

- Google colab
- Librería TensorFlow.
- Si tuliza Visual code instalar las librerías TensorFlow en Windows
    - `pip install tensorflow`
    - `pip install tensorflow-datasets`

Descripción

Implementación de una red neuronal densa regular para que logre escribir los números de la base de datos de MNIST.

Datos de entrada

1. Importar las librerías generales

```
import math
import numpy as np
import matplotlib.pyplot as plt
import logging
```

2. Importar librerías TensorFlow

```
import tensorflow as tf
import tensorflow_datasets as tfds
```

`import tensorflow as tf`: TensorFlow es una biblioteca de software de código abierto desarrollada por Google para realizar cálculos numéricos y construir modelos de aprendizaje automático y redes neuronales. Al importar TensorFlow como tf, se hace accesible toda la funcionalidad proporcionada por esta biblioteca, incluidas las operaciones para construir modelos, optimizadores, funciones de pérdida, funciones de activación, entre otras.

`import tensorflow_datasets as tfds:` TensorFlow Datasets es un módulo que proporciona un conjunto de herramientas para trabajar con conjuntos de datos comunes en el aprendizaje automático y la investigación de inteligencia artificial. Permite cargar, manipular y preprocesar fácilmente conjuntos de datos comunes, como MNIST, CIFAR-10, IMDB, etc. Importar **`tensorflow_datasets as tfds`** permite acceder a las funciones y conjuntos de datos proporcionados por TensorFlow Datasets.

3. Configuración del registro de TensorFlow, para mostrar mensajes de error, y así reducir el ruido de la salida de consola.

```
logger = tf.get_logger()
logger.setLevel(logging.ERROR)
```

`logger = tf.get_logger()`: Esta línea obtiene el registrador de TensorFlow. TensorFlow proporciona un sistema de registro incorporado que permite registrar mensajes de diferentes niveles (por ejemplo, información, advertencia, error) durante la ejecución del código. **`tf.get_logger()`** obtiene el objeto del registrador de TensorFlow.

`logger.setLevel(logging.ERROR)`: Esta línea establece el nivel de registro del objeto **`logger`** en **`logging.ERROR`**. Esto significa que solo se registrarán mensajes de error y niveles superiores (como advertencias y errores fatales). Los mensajes de información y depuración no se registrarán. Esto es útil cuando no se desea ver mensajes informativos o de depuración, y solo se quiere ver mensajes de error si ocurren.

4. Se obtienen los set de datos y metadatos del set, utilizando la librería **`mnist`** de **`tensorflow_datasets`** ría

```
dataset, metadata = tfds.load('mnist', as_supervised=True, with_info=True)
train_dataset, test_dataset = dataset['train'], dataset['test']
```

Se obtienen 60 mil datos de entrenamiento con **`train_dataset`** y 10 mil datos de prueba en **`test_dataset`**

5. Se agregan etiquetas simples de texto para los posibles resultados

```
class_names = [
      'Cero', 'Uno', 'Dos', 'Tres', 'Cuatro', 'Cinco', 'Seis', 'Siete',
      'Ocho', 'Nueve'
]
```

6. Se obtienen la cantidad de ejemplos en variables, para poder utilizarlos más adelante.

```
num_train_examples = metadata.splits['train'].num_examples #60 mil datos train
num_test_examples = metadata.splits['test'].num_examples #10 mil datos test
```

7. Se normalizan los números de los pixeles de 0 a 255, para que sean de 0 a 1, es decir, 0 para blanco y 1 para negro. Sólo se divide el número entre 255.

```
#Normalizar: Numeros de 0 a 255, que sean de 0 a 1
def normalize(images, labels):
    images = tf.cast(images, tf.float32)
    images /= 255
    return images, labels
```

Normalizar el conjunto de datos de entrenamiento y prueba es una práctica común en el aprendizaje automático y la visión por computadora por varias razones:

a) **Estabilidad del algoritmo de optimización**: Muchos algoritmos de optimización utilizados en el entrenamiento de modelos de aprendizaje automático funcionan mejor cuando las características de entrada están en una escala similar. La normalización ayuda a estabilizar estos algoritmos y a mejorar su convergencia.

b) **Rango de valores limitado**: Al normalizar los valores de píxeles de las imágenes en un rango de 0 a 1, se evita que ciertos píxeles dominen sobre otros en términos de magnitud, lo que puede conducir a una mejor convergencia del modelo y a una mejora en el rendimiento general.

c) **Facilita la inicialización de pesos**: En redes neuronales, la inicialización de pesos es una consideración importante para el entrenamiento efectivo del modelo. La normalización puede ayudar a mitigar problemas de inicialización al mantener los valores dentro de un rango más manejable, lo que puede facilitar el entrenamiento y evitar problemas como el desvanecimiento del gradiente.

d) **Generalización del modelo:** La normalización puede ayudar al modelo a generalizar mejor en datos no vistos, ya que reduce la sensibilidad a las diferencias en la escala de los datos. Esto puede llevar a modelos más robustos y mejorados en términos de rendimiento en conjuntos de datos desconocidos.

8. Se invocan a los métodos de normalización para cada set de datos, tanto *datos de entrenamiento* como *datos de prueba.*

```
train_dataset = train_dataset.map(normalize)
test_dataset = test_dataset.map(normalize)
```

9. En esta parte del código se **modela** la estructura de la **red neuronal** con una *capa de entrada* con 784 neuronas ingresadas en una matriz de 28 x 28, dos *capas ocultas* con 64 neuronas cada una y una *capa de salida* con 10 neuronas.

```
#Estructura de la red
model = tf.keras.Sequential([
  tf.keras.layers.Flatten(input_shape=(28,28,1)), #Capa de entrada
  tf.keras.layers.Dense(64, activation=tf.nn.relu), #Capas oculta
  tf.keras.layers.Dense(64, activation=tf.nn.relu), #Capas oculta
  tf.keras.layers.Dense(10, activation=tf.nn.softmax) #para clasificacion
])
```

Las activaciones **`relu`** se utilizan en las capas ocultas para introducir no linealidad en el modelo y acelerar el entrenamiento, mientras que la función **`softmax`** se utiliza en la capa de salida para generar probabilidades de clasificación.

10. Indicamos las funciones que se van a utilizar

```
#Función que compila el modelo
model.compile(
  optimizer='adam',
  loss='sparse_categorical_crossentropy',
  metrics=['accuracy']
)
```

Se compila el modelo especificando:

- El **optimizador** el cuál utiliza el algoritmo de optimización **Adam** ampliamente utilizado en el entrenamiento de redes neuronales debido a su eficacia y eficiencia.
- La **función de costo** a utilizar, para este problema se está utilizando la *función de pérdida de entropía cruzada categórica dispersa*. Esta función de pérdida es adecuada para problemas de clasificación multiclase, donde las etiquetas de clase son enteros y no necesitan ser codificadas en un formato one-hot.
- Las **métricas**, estas se calcularán y mostrarán durante el entrenamiento y la evaluación del modelo. En este caso, se está utilizando la métrica de *precisión*, que calcula la fracción de ejemplos de entrada cuyas etiquetas predichas coinciden exactamente con las etiquetas verdaderas. La *precisión* es una métrica comúnmente utilizada para evaluar el rendimiento de un modelo de clasificación.

```
#Aprendizaje por lotes de 32 cada lote
BATCHSIZE = 32
train_dataset = train_dataset.repeat().shuffle(num_train_examples).batch(BATCHSIZE)
test_dataset = test_dataset.batch(BATCHSIZE)
```

El bloque de código anterior, define el tamaño del lote que se utilizará durante el entrenamiento y la evaluación del modelo. El tamaño del lote es la cantidad de ejemplos de entrenamiento que se utilizan en una iteración del algoritmo de optimización durante el entrenamiento, para este problema se ordenan de forma aleatoria para garantizar que el modelo no aprenda patrones basados en el orden de los ejemplos de entrenamiento.

```
#Realizar el aprendizaje
model.fit(
  train_dataset, epochs=5,
  steps_per_epoch=math.ceil(num_train_examples/BATCHSIZE)
)
```

Función de **entrenamiento**, entrena el modelo utilizando el conjunto de datos de entrenamiento durante 5 épocas, realizando un número específico de pasos de entrenamiento en cada época. Durante el entrenamiento, el modelo ajustará sus pesos para minimizar la función de pérdida especificada durante la compilación.

11. Evaluación del entrenamiento

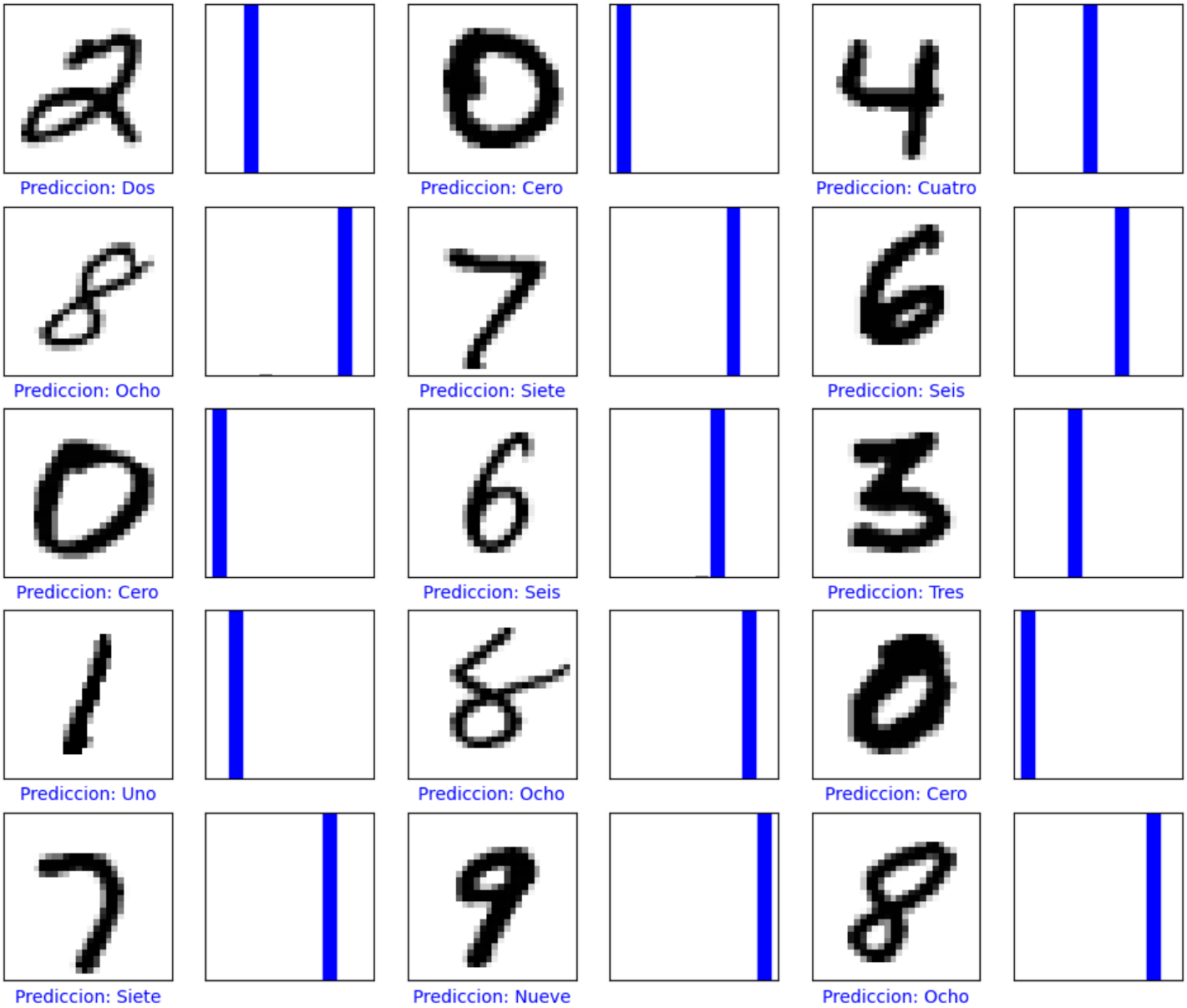
```
#Evaluar nuestro modelo ya entrenado, contra el dataset de pruebas
test_loss, test_accuracy = model.evaluate(
  test_dataset, steps=math.ceil(num_test_examples/32)
)

#Imprime los resultados de precisión
print("Resultado en las pruebas: ", test_accuracy)
```

Aquí se evalúa el modelo entrenado utilizando el conjunto de datos de prueba y calcula la pérdida y la precisión del modelo en el conjunto de datos de prueba. Los valores de pérdida y precisión se almacenan en las variables **`test_loss`** y **`test_accuracy`**, respectivamente.

12. Impresión de salidas en modo gráfico

```
for test_images, test_labels in test_dataset.take(1):
  test_images = test_images.numpy()
  test_labels = test_labels.numpy()
  predictions = model.predict(test_images)

def plot_image(i, predictions_array, true_labels, images):
  predictions_array, true_label, img = predictions_array[i], true_labels[i], images[i]
  plt.grid(False)
  plt.xticks([])
  plt.yticks([])

  plt.imshow(img[...,0], cmap=plt.cm.binary)

  predicted_label = np.argmax(predictions_array)
  if predicted_label == true_label:
    color = 'blue'
  else:
    color = 'red'

  plt.xlabel("Prediccion: {}".format(class_names[predicted_label]), color=color)

def plot_value_array(i, predictions_array, true_label):
  predictions_array, true_label = predictions_array[i], true_label[i]
  plt.grid(False)
  plt.xticks([])
  plt.yticks([])
  thisplot = plt.bar(range(10), predictions_array, color="#888888")
  plt.ylim([0,1])
  predicted_label = np.argmax(predictions_array)

  thisplot[predicted_label].set_color('red')
  thisplot[true_label].set_color('blue')
```

Resultado esperado:



Redes Neuronales – E1 – parte 2

Utilizando una página HTML para

Requerimientos para realizar el ejercicio

- Google colab.
- Librería TensorFlow.
- Redes Neuronales – E1 parte 1

Pasos

- Hacer la página HTML para escribir el número a mano alzada.
- Enviar con Ajax el dibujo hacia Python
- Agregar en Python soporte para recibir peticiones HTTP y procesarlas con la red neuronal

Puede utilizar cualquier código de ejemplo para escribir en html y escribir con el mouse.

Dependiendo del código que usted haya utilizado, ubique la función que determina cuando "deja de escribir" y dentro de ella realizar lo siguiente:

- Obtener los pixeles del canvas e ingresarlos en un arreglo (Recuerde que el arreglo es de 784 porque los pixeles se acomodan en una matriz de 28x28)

```
var pixels = [];
    for (var x=0; x < 28; x++) {
        for (var y=0; y < 28; y++) {
           var imgData = ctx.getImageData(y,x,1,1);
           var data = imgData.data;

           //Pixel negro o blanco?
           var color = (data[3])/255; //Data tiene 4 canales:Rojo, Verde, Azul, Alpha
                                      //Divido entre 255 para tener de 0 a 1

          //Dejar siempre 2 decimales
          color = (Math.round(color*100)/100).toFixed(2)
           pixels.push(color);
         }
    }
```

Ahora ese arreglo se envía por HTTP

```
<script
         src="https://code.jquery.com/jquery-3.4.1.min.js"
         integrity="sha256-CSXorXvZcTkaix6Yvo6HppcZGetbYMGWSFlBw8HfCJo="
         crossorigin="anonymous">
</script>
```

Se utiliza Ajax con JQuery (Puede utilizar el que mejor domine), usando el puerto 8000 en donde se coloca el servidor de Python.

```
$.post("http://localhost:8000", {pixeles: pixels.join(",")},
                function(response) {
                    console.log("Resultado: " + response);

                }
```

Retomando el código de Pyhton del ejercicio **E1 parte 1**,

1. Agregar las librerías

```
from urllib import parse
from http.server import HTTPServer, BaseHTTPRequestHandler
```

2. Eliminar las líneas de código que se encuentran después del código de entrenamiento *model.fit*

3. Crear una clase llamada `SimpleHTTPRequestHandler` para definir el servidor http, el cual sólo recibe solicitudes POST.

```
class SimpleHTTPRequestHandler(BaseHTTPRequestHandler):

    def do_POST(self):
        print("Peticion recibida")

        #Obtener datos de la petición y limpiar los datos
        content_length = int(self.headers['Content-Length'])
        data = self.rfile.read(content_length)
        data = data.decode().replace('pixeles=', '')
        data = parse.unquote(data)

        #Realizar transformación para dejar igual que los ejemplos que usa MNIST
        arr = np.fromstring(data, np.float32, sep=",")
        arr = arr.reshape(28,28)
        arr = np.array(arr)
        arr = arr.reshape(1,28,28,1)

        #Realizar y obtener la predicción
        prediction_values = model.predict(arr, batch_size=1)
        prediction = str(np.argmax(prediction_values))
        print("Prediccion final: " + prediction)
```

```
        #Regresar respuesta a la peticion HTTP
        self.send_response(200)

        #Evitar problemas con CORS
        self.send_header("Access-Control-Allow-Origin", "*")
        self.end_headers()
        self.wfile.write(prediction.encode())
```

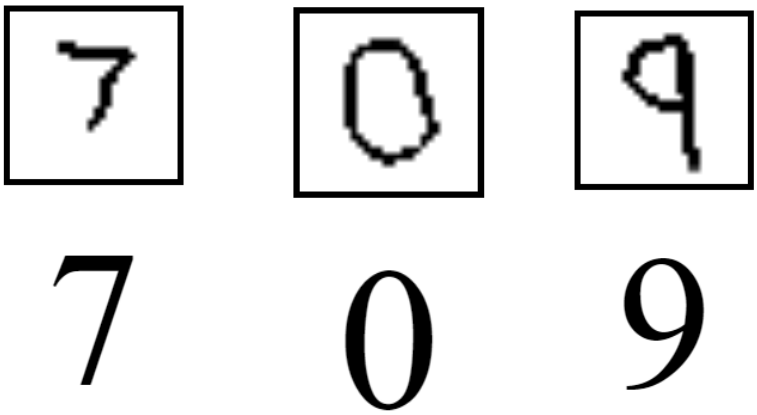
4. Iniciar el servidor

```
#Iniciar el servidor en el puerto 8000 y escuchar por siempre
#Si se queda colgado, en el admon de tareas buscar la tarea de python y
finalizar tarea
print("Iniciando el servidor...")
server = HTTPServer(('localhost', 8000), SimpleHTTPRequestHandler)
server.serve_forever()
```

5. Regresando al código de HTML, se toma el resultado generado en python y se visualiza la salida

```
$.post("http://localhost:8000", {pixeles: pixels.join(",")},
                function(response) {
                    console.log("Resultado: " + response);
                    $("#resultado").html(response); #resultado en el html
                }
```

Resultado esperado



Para visualizar más datos en MNIST consultar https://datasets.activeloop.ai/docs/ml/datasets/mnist/